Written assessment answers

Concise draft responses to all nine Part 1 questions.

Read these alongside the proposed Companies implementation. Adapt the wording to your own experience and submit only reasoning you can explain. The assessment allows AI assistance and requires you to defend every line.

1. Server and Client Component boundary

I use Server Components for the CRM page shell and server-rendered headings because they need no browser interaction and can keep server-only work out of the client bundle. The filterable table and detail drawer are Client Components because they use event handlers, URL hooks, TanStack Query and editable state. I place the client boundary around that interactive feature, passing only serializable data or messages across it, while keeping the layout and unrelated static content on the server. [1]

2. Server fetching and TanStack Query hydration

I would still fetch on the server when initial HTML needs the data, a server-only credential is required, or prefetching can remove a client request waterfall. I create a request-scoped QueryClient, prefetch using the same key/options as the client, then dehydrate it into a HydrationBoundary. The client hydrates that result with a suitable nonzero staleTime so mounting does not immediately fetch again; credentials stay outside the serialized cache. For this mock, I would keep company fetching client-side rather than add unnecessary SSR complexity. [2]

3. Version conflicts in UI and cache

On 409 I show that another person changed the company and keep the user's draft separate from the query cache, with no automatic mutation retry. I mark the detail stale and invalidate affected lists, while an explicit reload fetches the latest record without resetting the form. The user reviews their preserved edits against that record and explicitly resubmits against its version, starting a new edit baseline; any optimistic cache change would first be rolled back.

How these answers fit the practical build

Optimistic concurrency is a version check, not a requirement to render optimistic mutations. A simple pending save followed by authoritative cache updates is easier to defend within the timebox. Fetching in Server Components is explained in question 2 as a production option; it does not need to be added to Part 2.

Assessment source: nextjs-developer-test.pdf, supplied by the candidate. Page references below refer to its four pages.

4. URL state and query keys

The URL is the source of truth for committed search, status, sort and page; I normalize it once and use that same object for controls, API parameters and query keys. This makes refresh and browser Back/Forward restore the same results, and filters reset the page in the same URL update. For this feature I would commit search on Enter or Search, push meaningful navigation changes and keep only unfinished input locally; the drawer ID is separate URL state and does not affect the list key. [3, 4]

5. Shared access tokens and expiry

I inject an async token getter from the hosted-identity SDK into the shared client, so each request obtains a current access token and attaches its bearer header without components handling credentials. I deduplicate renewal when expiry occurs, allow at most one retry after a genuine unauthenticated response, and clear auth/cache and send the user to sign-in if renewal fails; I do not blindly replay writes after uncertain network failures. Tokens must never appear in URLs, logs, analytics, cache keys, source control or serialized page data, and production storage should follow the provider's guidance rather than default to localStorage. Part 2 intentionally uses only a fake stored demo token and redirects on 401. [5]

6. Loading, errors, empty states and boundaries

I use loading.tsx for route-segment loading and error.tsx for unexpected render failures, with Suspense around genuinely suspending subtrees and the useSearchParams client entry when prerendering requires it. The table and drawer have independent query loading, retry and empty/not-found states, because ordinary useQuery and mutation errors are handled explicitly rather than automatically by Suspense or route error boundaries. I keep the table and its controls mounted during filter refetches, using previous data and an updating indicator, and avoid a single boundary that blanks the whole screen whenever the drawer loads. [6, 7, 8]

Important distinctions to defend

A route error boundary does not catch every asynchronous request or event-handler error. A background detail refetch also does not grant permission to replace a dirty form or silently advance its concurrency version. Keep those behaviors explicit in the implementation and tests.

7. Dutch and English

I resolve one selected locale on the server, load only its dictionary through a server-only loader, and pass the required messages plus locale to client components. For this exact /companies route I would use a locale cookie instead of introducing language-prefixed URLs, and refresh the server-rendered messages after a toggle. Server and client formatting use the same locale and timezone through Intl.DateTimeFormat and Intl.NumberFormat, so dates and numbers agree without shipping both dictionaries to the client. [9]

8. Measuring and improving a slow page

I first reproduce a specific slow interaction in a production build and separate server/API latency, request waterfalls, JavaScript download/hydration and React rendering using browser traces and profiling. I check TTFB, LCP and INP alongside query timings, with cold/warm cache and representative network conditions, rather than treating the mock's intentional delay as a rendering defect. Two App Router features I would use are Server Components to reduce unnecessary client JavaScript and streaming through focused Suspense/loading.tsx boundaries so ready content appears while slower regions finish; I would verify the improvement against the same scenario. [1, 2, 10]

9. Testing strategy

I unit test pure URL normalization, query-key changes and validation because they encode small deterministic contracts. I component-test filtering through real controls and mocked HTTP, and the complete 409 sequence: original version sent, draft retained, latest reload, explicit resubmission and cache refresh. A browser E2E test is best for real routing/history, a shared drawer URL, auth redirects and focus trapping, which a simulated DOM cannot fully prove; under this timebox I prioritize the required component/unit tests and manually verify those browser behaviors.

References for technical claims

These are supporting references, not extra assessment requirements. Conflict UX, testing priorities and the proposed architecture are reasoned design choices.

[1] Next.js Server and Client Components
[2] TanStack Query advanced server rendering
[3] Next.js native History API
[4] TanStack Query keys
[5] Auth0 token storage guidance
[6] Next.js error handling
[7] Next.js useSearchParams
[8] TanStack Query previous-data behavior
[9] Next.js internationalization
[10] Next.js streaming and navigation